PayLink — midterm submission and internal knowledge-transfer guide

What to submit

The supplied assignment text requests **one video per team**, narrated by one representative explaining progress and each teammate's contributions. Only one team submission is needed. The materials do not specify an exact video duration or written-report length. Suggested internal format: a **7–9 minute video**, supported by the report, editable ER model, SQL, and validation evidence. This is a recommendation, not an invented course requirement.

The current artifact package contains the application source, backend, database migrations, synthetic local seed, ER diagram sources, normalization/design analysis, report, browser screenshots, tests, SQL exercises, and deployment instructions. **The live public website and hosted Supabase verification are still pending account/tool access.** Do not record a hosted-success claim until the deployment acceptance checklist passes.

Contribution split: assigned responsibility, pending team confirmation

The implementation was produced with Codex assistance in this session. The following is a balanced **recommended ownership split** for review, explanation, follow-up changes, and the video. It is not a fabricated historical record of authorship. Each teammate should run the work, explain the design, make/record their actual contributions, and update the final attribution.

Area	Sai — proposed responsibility	Siddhardth — proposed responsibility	Evidence to prepare
Requirements and modeling	Financial rules, FDs/candidate keys, account/transfer constraints	ER diagram, cardinalities, joint ownership, relational translation	Explain keys, bridge table, and source/destination roles together.
Database implementation	Five-table DDL, restricted role, transfer locks/idempotency, migrations	Seed/catalog review, dictionary, advanced SQL joins/aggregation/window examples	Run migrations and queries; show the schema and no duplicated bank facts.
Application	FastAPI routes, Argon2id/session identity, DB calls, exception/rollback handling	Next.js/TypeScript views, responsive styling, account/recipient/transfer flows, CSV and insights	Demonstrate identity-to-ownership validation and UI-to-DB data flow.
Validation and delivery	Concurrency/integrity tests, reconciliation, index/EXPLAIN investigation	Browser E2E, visual/mobile checks, screenshots, report/video, deployment configuration	Review test outputs; capture the public app and actual Supabase results after access works.

Suggested speaking balance if one representative records: name both teammates at the start, then explicitly credit each assigned/confirmed area. If Sai narrates, use “Siddhardth owns…” only after he confirms it; if Siddhardth narrates, mirror the wording for Sai. Replace proposed assignments with actual verified work before submission.

Four incremental milestones

1. **Domain and design.** Agree on simulation scope, five entities, many-to-many joint ownership, candidate keys, and an editable ER diagram. Deliverables: proposal alignment, relationship model, FD/normalization notes. Exit criterion: both teammates can justify why ownership is a separate relation.
2. **Reliable data layer.** Implement versioned DDL, keys/constraints, synthetic sample data, account opening, and the transfer function. Deliverables: recreation scripts, dictionary, advanced SQL examples. Exit criterion: clean migration, correct FK/uniqueness rules, and reconciliation with no differences.
3. **End-to-end application.** Integrate FastAPI authentication and database access with Next.js accounts, recipient lookup, transfer review, history, CSV, and insights. Exit criterion: a signed-in user can transfer and both parties see the persisted result.
4. **Validation and midterm demonstration.** Run integrity/concurrency/rollback tests, examine index plans, capture desktop/mobile evidence, and assemble the report/video. Local checks have been performed. **The required public deployment and hosted Supabase acceptance portion is pending**; do not mark the entire milestone complete until the live site and DB are verified.

Follow-on milestones 5, 6, and 7

- **5 — Hosted completion and operations:** connect Supabase primary-org/paylink, deploy, run isolated hosted acceptance, capture UI plus Supabase evidence, and test a backup/restore procedure. Resolving the existing deployment blocker is the first task; it is not optional future polish.
- **6 — Model and security extensions:** immutable double-entry ledger/audit model, reversals, pagination, joint-owner invitations/revocations, stronger identity lifecycle, and explicit revised invariants.
- **7 — Performance and final evaluation:** representative load, workload skew, repeatable query-plan comparisons, multi-worker validation, accessibility refinement, and final course presentation.

Video script and screen cues (approximately 8 minutes)

Use this as a rehearsal script; shorten or adapt to the instructor’s actual requirements. Bracketed text is an instruction to the presenter, not spoken content. Do not read unverified authorship or deployment claims.

0:00–0:25 — Introduction

“Hello, this is our midterm progress update for CS 5600 Advanced Database Systems. I’m [presenter’s name], and I’ll walk through our project, its database design, our implementation progress, and the next milestones.”

[Show the title slide or report cover.]

0:25–0:50 — Teammate introductions and project title

“Our team is Sai Kadamati and Siddhardth. The project is **PayLink: Simulated Peer-to-Peer Banking Transfers**. Sai’s assigned focus is the database transaction and backend layer. Siddhardth’s assigned focus is the interface, relationship documentation, and browser demonstration. We share design review and validation.”

[Replace “assigned focus” with actual confirmed contributions if the team has completed and documented them. Include the AI-assistance acknowledgment required by the course.]

0:50–1:25 — Project description

“PayLink lets a user open simulated accounts, find a recipient by username, email, or phone, send money, move money between owned accounts, and review activity. It also supports joint accounts. No real bank credentials or real funds are involved. The database problem is keeping both balances and the transfer record correct when requests overlap, fail, or are retried.”

[Show the overview, then point out the simulation label and shared account.]

1:25–2:20 — Four milestones

“Our first milestone defined the domain and ER model. The second implemented the relational schema, constraints, sample data, and transaction rules. The third connected a FastAPI backend to a Next.js interface. The fourth covers tests, query plans, screenshots, and the midterm demonstration. We have local implementation evidence for these stages. Hosted Supabase and the public website are [pending / verified as shown next].”

[Choose the truthful status. Show the milestone table. Explain Sai’s and Siddhardth’s confirmed work rather than merely displaying names.]

2:20–3:30 — Major database design and normalization

“There are five core tables: Users, Banks, BankAccounts, AccountHolders, and Transfers. Banks to Accounts is one-to-many. Users to Accounts is many-to-many, so AccountHolders uses a composite user/account key. This lets two users share one account balance without copying the account.

“We separated user, bank, account, ownership, and transfer facts to remove partial and transitive dependencies. For example, an account determines its bank ID, while the bank ID determines bank details. Storing a bank name on every account would repeat those facts. In the final relations, each declared nontrivial functional dependency starts from a candidate key, giving BCNF under our assumptions. Ownership and sufficient funds are transaction rules, not something normalization alone guarantees.”

[Open the editable diagram, point to the bridge and the two account roles on Transfers, then show the FD table.]

3:30–4:35 — Atomicity, concurrency, and duplicate prevention

“A transfer checks the signed-in actor’s source ownership and locks both account rows in a consistent UUID order. It checks active state and funds before debiting one account, crediting the other, and inserting

the event within one transaction. If any step fails, the whole transaction rolls back.

“A user-scoped idempotency key prevents a retry from sending the same payment twice. An identical retry returns the original transfer; a changed payload with the same key is rejected. Our tests also check competing spends, opposing transfers, joint owners, and direct balance-update permissions.”

[Show the transfer function excerpt and current test result. Explain READ COMMITTED plus row locks; do not claim arbitrary serializable behavior.]

4:35–5:10 — SQL and query processing

“We use joins and aggregation for account totals and shared ownership, a window function for running balances, and a reconciliation query comparing stored balances against opening balances and transfer deltas. The local reconciliation found zero differences. A 20,000-row temporary microbenchmark compared account history with and without the composite index: the indexed plan avoided scanning and sorting the full fixture. These local timings are illustrative, not a production performance promise.”

[Show the query and actual EXPLAIN output. The fixture and index drop were rolled back.]

5:10–6:45 — Basic UI and database demo

[For the final requested video, use the **actual public website** and the **actual Supabase project** after deployment. Until then, local rehearsal is only a rehearsal.]

“I’ll sign in as Sai. Here are the simulated accounts, including one jointly held account. I’ll send $12.34 to Siddhardth. The review screen shows the source, destination, amount, and remaining balance. After confirmation, the receipt gives us a transfer ID.

“Now we can use that exact ID to inspect the row in Supabase. We should see one transfer, a source balance lower by 1,234 cents, and the destination higher by the same amount. Reloading the app preserves the payment. In Siddhardth’s session, it appears as received. An insufficient-funds request is rejected and leaves the balances unchanged.”

[Have the source and destination balances recorded **before** sending. Show only the relevant rows and no credentials. Run the read-only reconciliation query; zero rows means no discrepancies. Show the mobile screenshot briefly if time permits.]

6:45–7:30 — Contributions and progress recap

“[Sai’s confirmed work and evidence]. [Siddhardth’s confirmed work and evidence]. The integration tests and browser checks cover the important correctness paths. [State hosted completion or the remaining deployment blocker accurately].”

[Each teammate should supply one concrete example they can explain: a locking decision for Sai, and a UI-to-database/browser assertion or relationship/query decision for Siddhardth.]

7:30–8:10 — What’s next: milestones 5–7

“Milestone five completes hosted acceptance and operational checks, including live Supabase evidence and a recovery exercise. Milestone six extends the account and audit model with reversals, owner

management, and a stronger ledger. Milestone seven focuses on representative performance tests, query optimization, accessibility, and the final evaluation. Thank you.”

[If hosted acceptance is already finished, say that this part of milestone five is complete and identify the next operational checks.]

Recording checklist

- One teammate records and one representative submits one video for the team.
- Confirm the instructor permits this AI assistance and include an accurate disclosure.
- Review the report and edit contribution assignments to actual work.
- Use the stable public deployment; avoid recording free-tier cold-start delays.
- Use only synthetic data and avoid displaying cookies, password fields, connection strings, or secret panels.
- Keep two signed-in sessions available, one for each user; shared cookies in one browser profile will replace the prior session.
- Record balances before/after a transfer; show the exact transfer ID in Supabase and the app.
- Rehearse the insufficient-funds rejection and idempotency explanation.
- Confirm audio, readable zoom, and the instructor’s accepted video format/upload destination.
- Keep the editable diagram and source report as supporting materials; submit extra files only if requested by the assignment.

Knowledge-transfer questions for both teammates

1. Why is `user_id` not a column on BankAccounts? What anomaly would that cause for a joint account?
2. Why are money values cents rather than floating point?
3. What is the difference between a PK, alternate key, FK, and the idempotency key pair?
4. Which rules come from SQL constraints and which require the transaction function?
5. What happens if two payments try to spend the same balance? Why lock both rows in one order?
6. Why does nullable phone uniqueness not make it an unconditional candidate key?
7. What does the running-balance query prove, and what does zero-row reconciliation mean?
8. Why can the browser not receive `paylink_api` credentials? What does SECURITY DEFINER change?
9. Which claims are locally tested and which require live Supabase validation?
10. What did each teammate actually contribute, and which parts were generated with assistance?